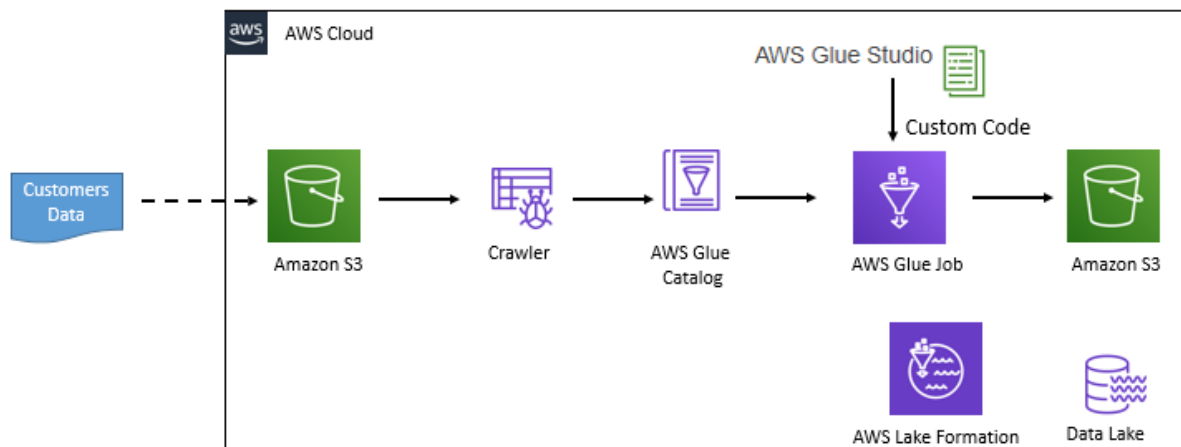


Exercise-2_Using Custom Transformation in AWS Glue Studio

AWS Glue Studio is GUI based service to create, run, and monitor extract, transform, and load (ETL) jobs in AWS Glue. It helps in visually composing data transformation workflows and run them on AWS Glue's Apache Spark-based serverless ETL engine. AWS Glue Studio supports both tabular and semi-structured data. AWS Glue Studio also offers tools to monitor ETL workflows and validate that they are operating as intended.

Custom Transform (custom code node) in AWS Glue Studio allows to perform complicated transformations on the data using custom code. The Custom code node allows to enter a script that performs the transformation on the data.

In this exercise, you create an ETL job using AWS Glue Studio which reads data from the data lake data catalog, performs transformation using custom code (Custom Transform) and writes to the S3 bucket.



1: Pre-requisite

You need to have an AWS account with required permissions access to complete the exercise. Check with your instructor.

2: Data and Processing

Before you start, let's understand the data and what data processing we configure by creating AWS Glue Job in AWS Glue Studio.

You will configure a quick and small data lake with a sample **customers** data. The data is stored in Amazon S3. AWS Glue and AWS Lake Formation services are used to create the data lake.

The following is the schema of the **customers** data:

Exercise-2_Using Custom Transformation in AWS Glue Studio

Fields: {CUSTOMERID, CUSTOMERNAME, EMAIL, CITY, COUNTRY, TERRITORY, CONTACTFIRSTNAME, CONTACTLASTNAME}

You then create a Glue job in the Glue Studio which performs the following transformation using **Custom Transformation** -

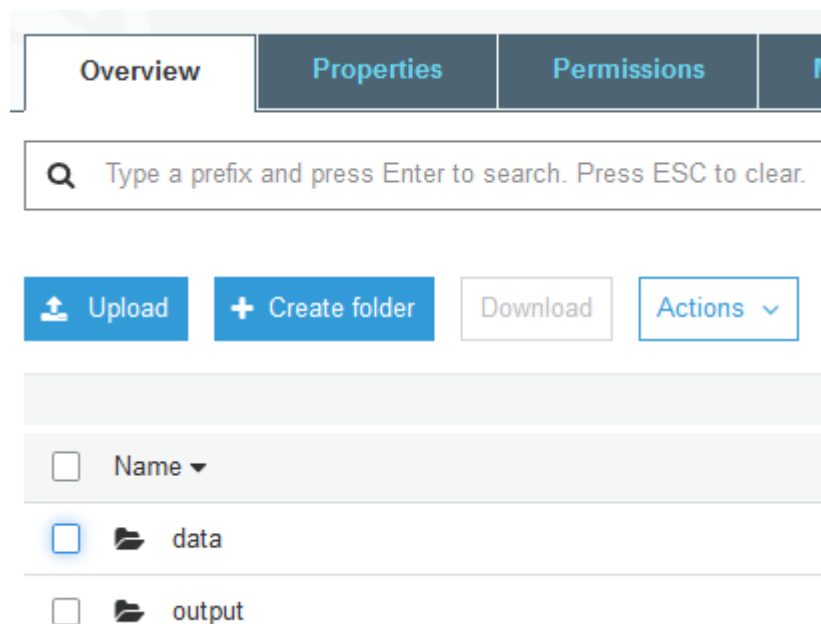
1. Concatenate **CONTACTFIRSTNAME** and **CONTACTLASTNAME** fields into **CONTACTFULLNAME** field.
2. Drop **CONTACTFIRSTNAME** and **CONTACTLASTNAME** fields.

Finally, the job writes the transformed data to the S3 bucket.

3: Create S3 Bucket

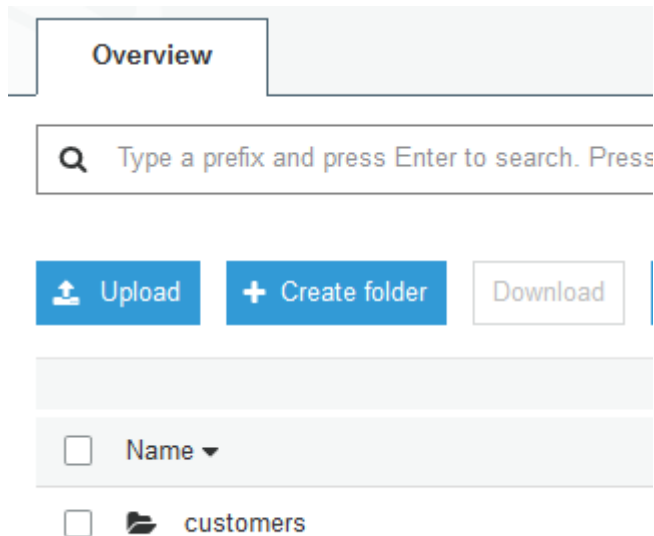
You create Amazon S3 bucket and upload data for the data lake. For this workshop, you are uploading data manually but in actual production usage the data is uploaded using data ingestion services / pipeline such as AWS Glue, Amazon Kinesis etc. Since the focus of the workshop is introduction to AWS Glue Studio, it keeps the data ingestion out of scope.

1. Goto S3 Management console, use + **Create bucket** button to create a new bucket with name **glue-customer-data**. If the bucket name is not available, then use a different name which is available.
2. Click on the **glue-customer-data** bucket to open it. Within the **glue-customer-data** bucket, create two folders **data** and **output** using the + **Create folder** button. The **data** folder is used to keep the data lake data while the **output** folder is used by the Glue Job to write processed data.

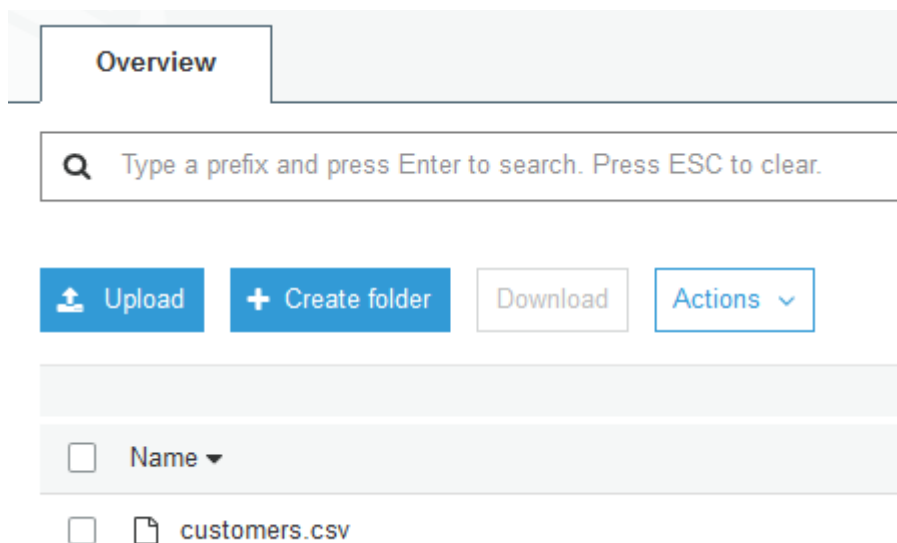


Exercise-2_Using Custom Transformation in AWS Glue Studio

- Click on the **data** folder to open it. Within the **data** folder, create a folder **customers** using the + **Create folder** button.



- Click to open **customers** folder and upload **customers.csv** file. Use **Upload** button to upload the files. The **customers.csv** file is available for download using the following link - [customers.csv](#)



- The data is ready. Let's start with the data lake configuration.

4: Configure Data Lake

The data is ready in Amazon S3 bucket. Let's start the configuration of the data lake. In data lake, you create a database and configure it with Amazon S3 bucket location. A database is used to organize data catalog tables in the data lake. You will later configure crawler to automatically create the data catalog tables in the data lake.

Exercise-2_Using Custom Transformation in AWS Glue Studio

1. Open the AWS Lake Formation console. If you are using Lake Formation for the first time in the region, it will ask you to create a data lake administrator. A data lake administrator is an IAM user or IAM role that performs administrative tasks in the data lake. For the first time user, it will popup a message to add administrators. Click on the **Add administrators** button to create administrators for your Data Lake.

Welcome to Lake Formation ×

The first step in creating your data lake in Lake Formation is to define one or more administrators. Administrators have full access to the Lake Formation system, and control the initial data configuration and access permissions.

Click Add administrators below to add your administrators.

Cancel Add administrators

2. Select **your AWS logged-in IAM user** from the drop down list. For the rest of handson, the user will be considered as a data lake administrator and will have full access to the data lake. Click on the **Save** button.

Manage data lake administrators ×

IAM users and roles
Add or remove IAM users and roles from the data lake administrators list.

Choose IAM principals to add ▼

×

User

Choose up to a maximum of 10 data lake administrators.

Active Directory users and groups (EMR beta only)
Enter one or more Active Directory users or groups.

Ex: `arn:aws:iam::<AccountId>:saml-provider/<SamlProviderName>`

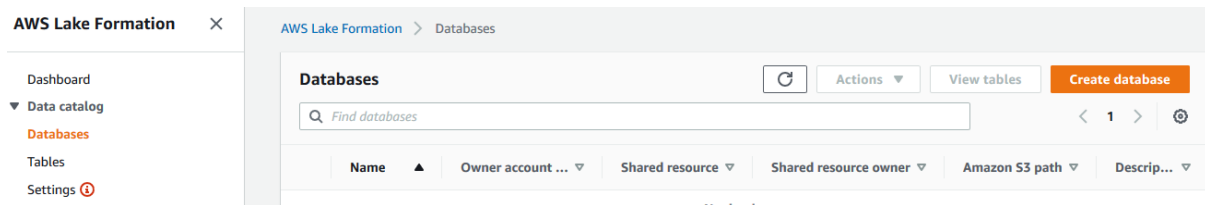
Cancel Save

Note: if you did not get the popup then that means the data lake already has an administrator. You can check that by clicking on the “Admins and database creators” menu in the left. If you see that your logged-in IAM username is listed as the “Data lake Administrator” then you are ok to move to the next step. Otherwise, click on the

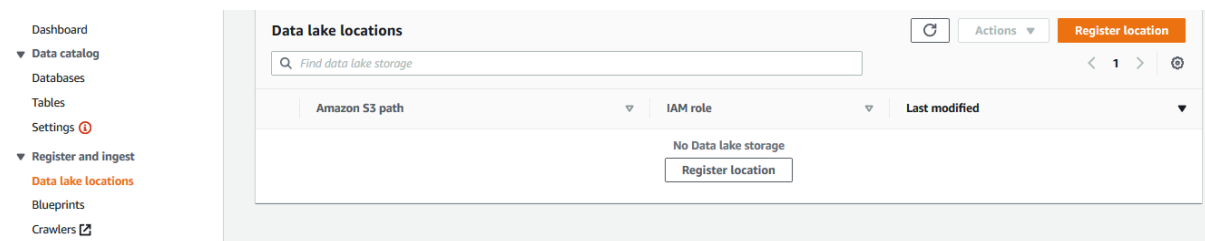
Exercise-2_Using Custom Transformation in AWS Glue Studio

“Grant” button to add “your AWS logged-in IAM user” as the administrator of the data lake.

3. After adding the administrator, you will create the database. In the AWS Lake Formation console, click on the **Databases** option on the left menu and then click on **Create database** button.



4. On the next screen, select **Database** option and type in **gluedb** as the **Name**. In the **Location** box, select the S3 data lake path as **s3://glue-customer-data/data**. If you created the bucket with different name, then you replace **glue-customer-data** part with that name. Make sure you **Uncheck** the option - **Use only IAM access control for new tables in this database**. Leave rest of the options as default and click on **Create database** button.
5. The database is added in no time. Now register the Amazon S3 bucket as your data lake storage. In the navigation pane, choose **Data lake locations**, and then choose **Register location**.



6. Enter a path to the existing S3 bucket **s3://glue-customer-data/data**. If you created the bucket with a different name, then you replace **glue-customer-data** part with that name. For the IAM role, select the **AWSServiceRoleForLakeFormationDataAccess** role. Click **Register location** to save it.
7. It is time to use crawler to catalog the data in the database.

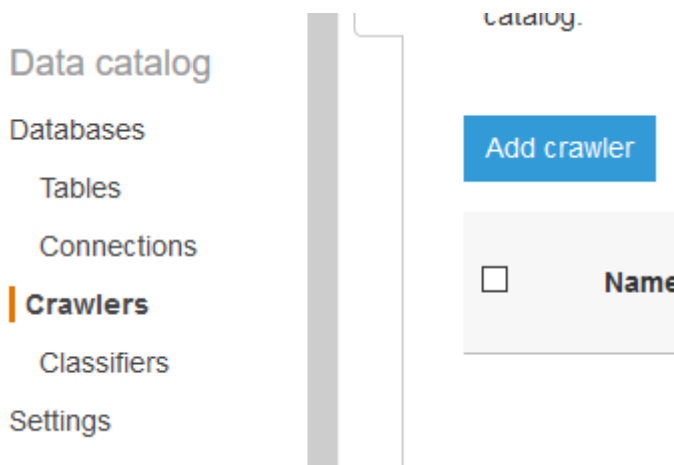
5: Configure and Run Crawler

One of the fundamental principle of building the data lake is that every data in the data lake should be catalogued. The catalog is automated using crawlers in AWS Glue. The crawler uses role based authorization to create catalog in the data lake database.

Open the AWS Lake Formation console, click on the **Databases** option in the left. You will see **gluedb** database listed.

Exercise-2_Using Custom Transformation in AWS Glue Studio

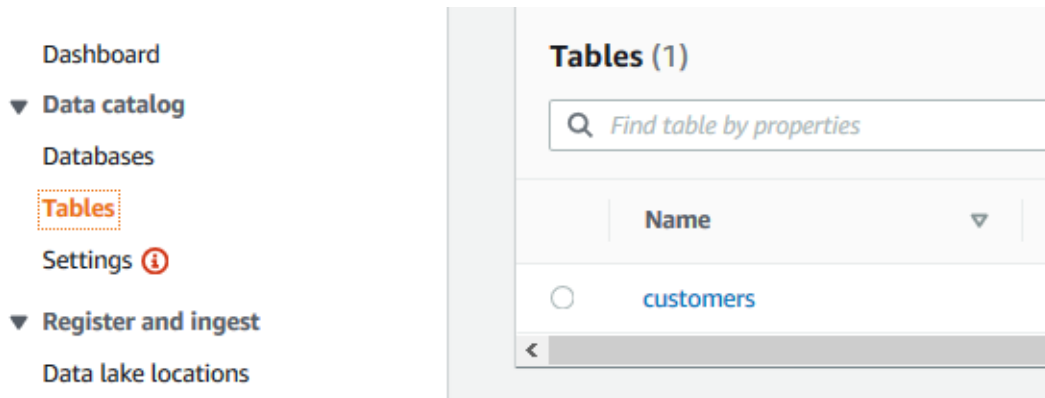
1. Select the **gluedb** database and click on the **Grant** menu option under the **Action** dropdown menu.
2. On Grant permissions screen, choose for the **My account** option and select **<role>** for the IAM users and roles field. Select only **Create table** and **Alter** permissions for the Database permissions. Then click on the **Grant** button. It means you are authorizing crawler role to be able to create and alter tables in the database.
3. After assigning permission, time to configure and run crawler. Open the AWS Glue console. Click on the **Crawlers** menu on the left and then click on the **Add crawler** button.



4. On the next screen, enter **gluecrawler** as the Crawler name and click **Next**.
5. On the next screen, select **Data stores** as the Crawler source type and click **Next**.
6. On the next screen, select **S3** as a data store and select **Specified path in my account** option. Provide the include path for the S3 bucket as **s3://glue-customer-data/data/customers**. If you created the bucket with a different name, then replace **glue-customer-data** part with that name. Click **Next**.
7. On the next screen, you are going to crawl only one data store, so select **No** from the option and click **Next**.
8. On the next screen, select **Choose an existing IAM role** option and then select IAM role from the dropdown list. Click **Next**.
9. On the next screen, select **Run on demand** as the frequency and click **Next**. In actual production use, you generally schedule crawler run so that it automatically updates the catalog.
10. On the next screen, select **gluedb** as the database. Click **Next**.
11. On the next screen, verify all crawler information on the screen and click **Finish** to create the crawler.
12. The crawler is created in no time. Select the crawler and click on the **Run crawler** button to run the crawler.

Exercise-2_Using Custom Transformation in AWS Glue Studio

13. It might take couple of minutes for the crawler to finish crawling the bucket. You would see a success message that there is one table **customers** created by the crawler in **gluedb** database.
14. Go back to the AWS Lake Formation console, click on the **Tables** menu in the left. You can see **customers** table created.



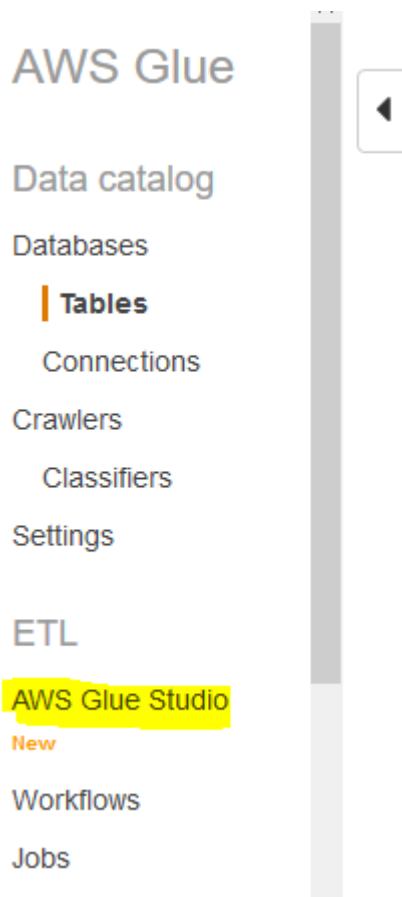
15. In the next step, you write a Glue Job using AWS Glue Studio.

6: Create Job in Glue Studio

In this step, you create a job using Glue Studio which reads data from the **customers** table and writes only {CUSTOMERNAME, EMAIL} fields to a S3 bucket folder.

1. First you grant **<role>** role **Select** access permission on the **customers** table because this role is used by the job you are going to create.
2. Go to Lake Formation Console. Click on the **Tables** menu in the left. Then select **customers** table and click on the **Grant** menu option under **Actions**.
3. On the next screen, select **My account** option. Select **<role>** as the IAM Users and roles. Choose **Select** for the table permission. Keep rest of the configuration to the default and click on the **Grant** button.
4. The role now has the required access permission. Go to Glue Service console and click on the **AWS Glue Studio** menu in the left.

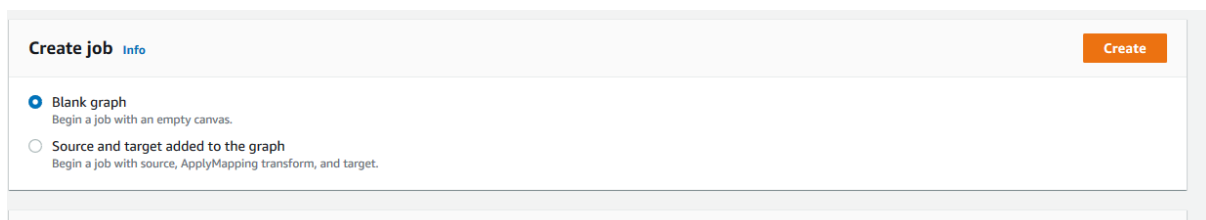
Exercise-2_Using Custom Transformation in AWS Glue Studio



5. On the next screen, click on the **Create and manage jobs** link.



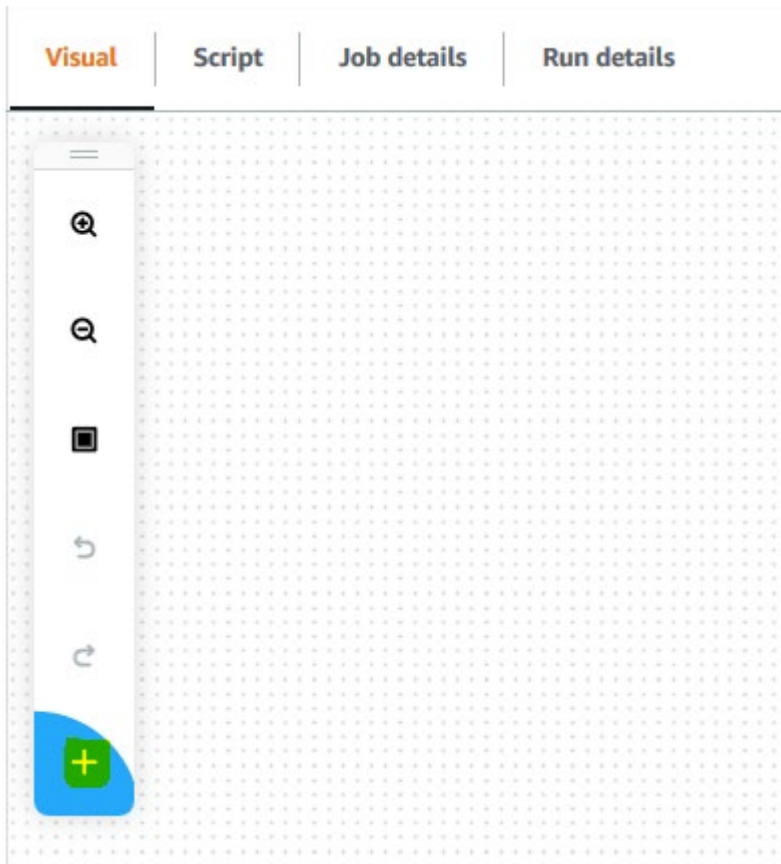
6. On the next screen, select **Blank graph** option and click on the **Create** button.



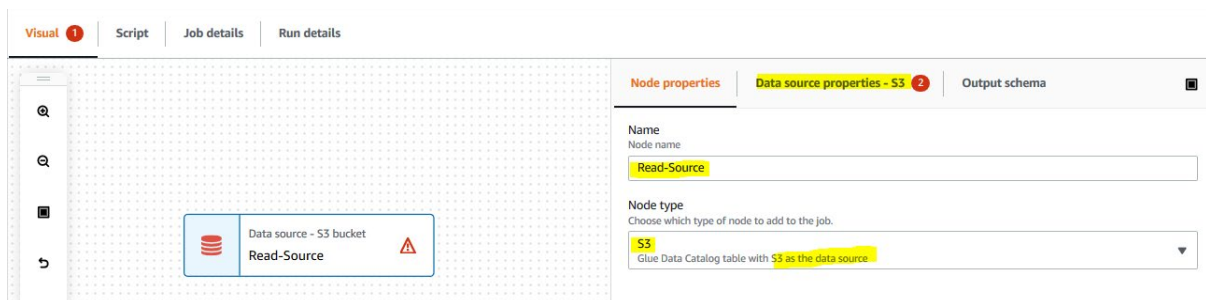
7. It opens the Glue Studio editor. Click on the **Job Details** tab. Type in **mygluejob** for the name and select **<role>** for the IAM Role. Keep rest of the fields as the default and click on the **Save** button.

Exercise-2_Using Custom Transformation in AWS Glue Studio

8. The job name and role configuration is saved. Go back to the **Visual** tab and click on the + icon.

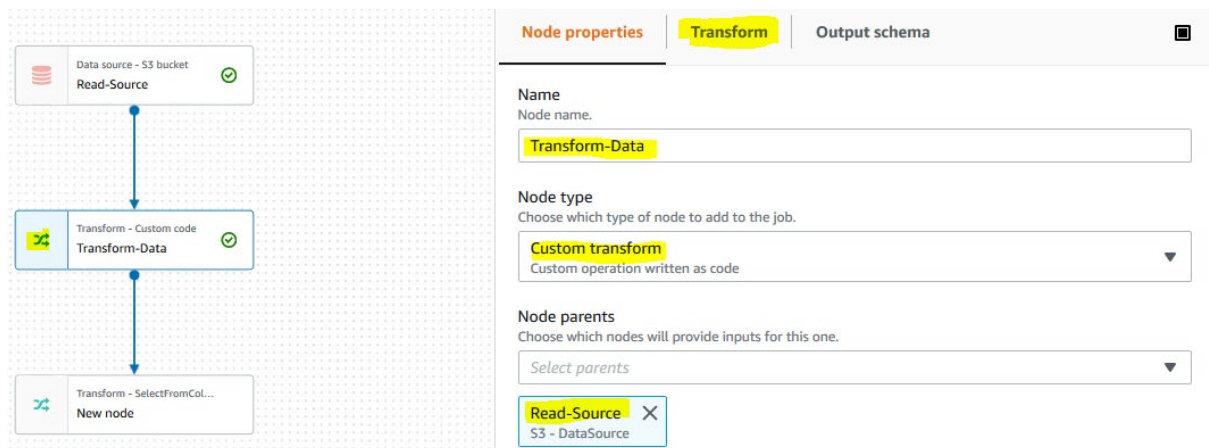


9. It creates a new node in the editor. On **Node properties** tab, type in **Read-Source** for the name. Select **Data source - S3 Bucket** for the node type. Click on the **Data source properties - S3** tab.



10. On the **Data source properties - S3** tab, select **gluedb** as the database, select **customers** for the table. You are selecting **customers** table in the **gluedb** as the source. Click on the **Save** button. The job is saved.
11. On the **Visual** tab and click on the + icon to add a new node. On **Node properties** tab, type in **Transform-Data** for the name. Select **Transform - Custom code** for the node type. When you select the node type as **Transform - Custom code**, it automatically adds another child node of type **Transform - SelectFromCollection** to allow to select the DynamicFrame from the output (DynamicFrameCollection) of the **Transform - Custom code** node. Keep the node parents field to **Read-Source**. Then click on the **Transform** tab.

Exercise-2_Using Custom Transformation in AWS Glue Studio



12. On the **Transform** tab, you can see a code block with function definition **def MyTransform (glueContext, dfc) -> DynamicFrameCollection**. Copy-paste the following code for the function definition to perform transformation on the data.

```
Node properties | Transform | Output schema

Code block Info
Enter a custom script to add to your job.

1 def MyTransform (glueContext, dfc) -> DynamicFrameCollection:
2     newdf = dfc.select(list(dfc.keys())[0]).toDF()
3
4     from pyspark.sql import functions as sf
5     newdf = newdf.withColumn('contactfullname', sf.concat(sf.col('contactf
6     newdf = newdf.drop('contactfirstname')
7     newdf = newdf.drop('contactlastname')
8
9     newcustomerdfc = DynamicFrame.fromDF(newdf, glueContext, "newcustomerdfc")
10    return (DynamicFrameCollection({"CustomTransform0": newcustomerdfc}, glueContext))
```

```
newdf = dfc.select(list(dfc.keys())[0]).toDF()

from pyspark.sql import functions as sf
newdf = newdf.withColumn('contactfullname',
sf.concat(sf.col('contactfirstname'),sf.lit(' '), sf.col('contactlastname')))
newdf = newdf.drop('contactfirstname')
newdf = newdf.drop('contactlastname')

newcustomerdfc = DynamicFrame.fromDF(newdf, glueContext, "newcustomerdata")
return (DynamicFrameCollection({"CustomTransform0": newcustomerdfc},
glueContext))
```

13. The code is simple. The input parameter of **DynamicFrameCollection** type has one **DynamicFrame** which was fetched in the **Read-Source** node. You collect the **DynamicFrame** using **index** value of **0**, convert to **DataFrame** and assign to the **newdf**

Exercise-2_Using Custom Transformation in AWS Glue Studio

variable. The code then uses `pyspark.sql.functions` to concatenate `contactfirstname` and `contactlastname` fields into a new field `contactfullname`. You also drop `contactfirstname` and `contactlastname` fields from the dataframe. You again assign the transformed dataframe to the `newdf` variable. Finally you convert `newdf` dataframe into `newcustomerdyd` DynamicFrame and add to the DynamicFrameCollection. The collection is then returned.

14. After finishing the code, click on the **Output schema** tab. You need to modify the output schema in order to match with the transformation in the code. You Remove `contactlastname` field and rename `contactfirstname` field to `contactfullname` to match the output schema of the transformation. Finally click on the **Apply** button.

Key	Data Type	Add	Delete
customerid	long		
customername	string		
email	string		
city	string		
country	string		
territory	string		
contactfullname	string		

15. Save the job. Select the next **Transform - SelectFromCollection** node which was automatically added along with the **Transform - Custom code** node. Click on the **Transform** tab. Make sure the value in the **Frame** is **0**. This will ensure the only DynamicFrame in the DynamicFrameCollection from the **Transform - Custom transform** node gets selected.

Node properties | **Transform** | Output schema

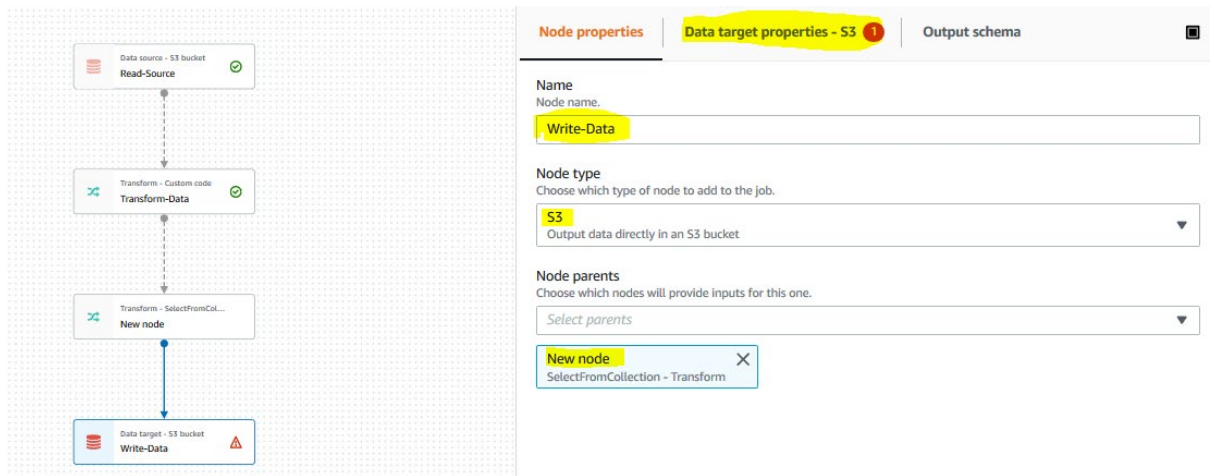
Frame
Index of the frame to select from the collection.

0

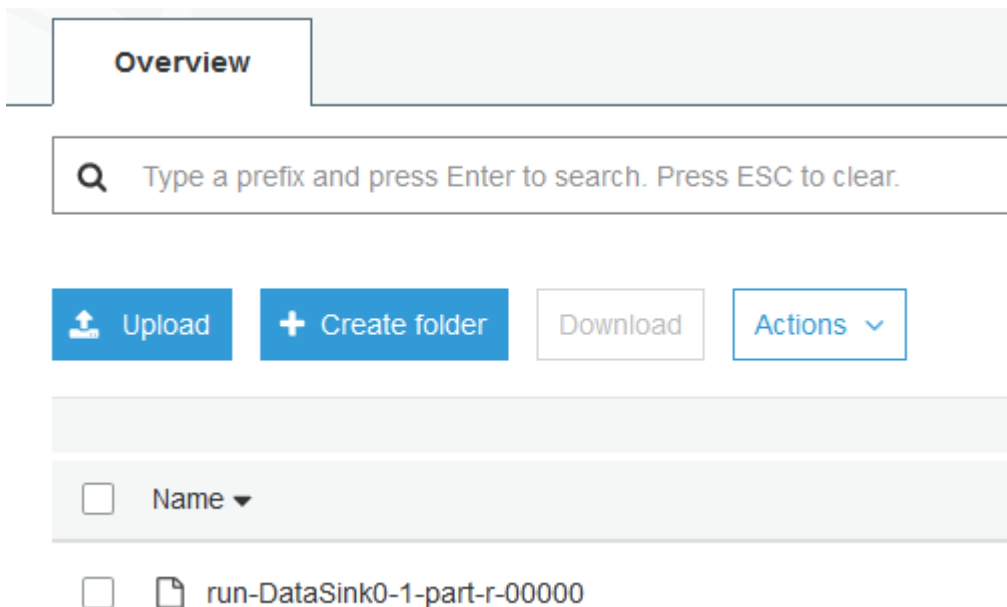
16. On the **Visual** tab and click on the + icon to add a new node. On **Node properties** tab, type in **Write-Data** for the name. Select **Data target - S3 Bucket** for the node

Exercise-2_Using Custom Transformation in AWS Glue Studio

type. Keep the node parents field to **Transform - SelectFromCollection** node. Then click on the **Data target properties - S3** tab.



17. On the **Data target properties - S3** tab, keep the format for **JSON**. Keep compression type to **None**. Select **s3://glue-customer-data/output/** for the S3 target location. If you created the bucket with a different name, then you replace **glue-customer-data** part with that name. Click on the **Save** button. You are asking Job to write transformed data to the S3 bucket in non-compressed JSON format. The job is saved.
18. The job is ready. Click on the **Run** button for the job. It will start the job execution. Click on the **Run Details** link to check the job status.
19. Wait till the status of the job changes to **Succeeded**.
20. You can go to the output S3 bucket location to see a new file created by the Glue Job.



21. If you download and open the file, you can see the output JSON data. You can see the data transformation completed.

Exercise-2_Using Custom Transformation in AWS Glue Studio

```
{ "customerid":1, "customername": "Land of Toys  
Inc.", "email": "gladys.rim@rim.org", "city": "NYC", "country": "USA", "territory": "NA", "contactfullname": "James Butt" }  
{ "customerid":2, "customername": "Reims  
Collectables", "email": "yuki_whobrey@aol.com", "city": "Reims", "country": "France", "territory": "EMEA", "contactfullname": "Josephine  
Darakjy" }  
{ "customerid":3, "customername": "Lyon  
Souvenirs", "email": "fletcher.flosi@yahoo.com", "city": "Paris", "country": "France", "territory": "EMEA", "contactfullname": "Art  
Venere" }  
{ "customerid":4, "customername": "Toys4GrownUps.com", "email": "bette_nicka@cox.net", "city": "Pasadena", "country": "USA", "territory": "  
NA", "contactfullname": "Lenna Paprocki" }
```